

Predictive Policing: Complete Project Review Handbook

Generated on: 2026-04-19 09:21:27

This document contains model metrics, full formulas, proof values, dashboard graph logic, and viva questions in simple words.

A) Evidence Snapshot

- Primary proof files used for all numbers in this report:
- 1) server/retrain_stdout.json
- 2) server/evaluate_metrics_stdout.json
- 3) server/latency_benchmark_stdout.json
- Code proof files:
- - server/retrain_model.py
- - server/tools/evaluate_hotspot_metrics.py
- - server/tools/benchmark_prediction_latency.py

B) Which Clustering Is Used and Why

- Algorithm: KMeans clustering on geospatial coordinates (latitude, longitude).
- Code proof: server/retrain_model.py:14, server/retrain_model.py:475
- Cluster count is selected using silhouette score on candidate k values.
- Code proof: server/retrain_model.py:458, server/retrain_model.py:626, server/retrain_model.py:629
- Why KMeans here: centroid output is easy to show as hotspot centers on map and fast for prediction-time checks.

C) Native Model Metrics (from retraining output)

- Records fetched: 137
- Records used: 134
- Records skipped: 3
- Coverage percent: 97.81%
- Direct coordinate hits: 132
- Geocoded hits: 2
- Cluster count: 5
- Cluster sizes: {'0': 127, '1': 3, '2': 1, '3': 2, '4': 1}
- Silhouette score: 0.9849
- Model inertia: 0.0001
- Avg cluster spread (km): 0.004

D) Accuracy, Precision, Recall, F1 with Full Proof

- These are computed through temporal holdout + spatial hit-testing (proxy evaluation for unsupervised model).
- Code proof for formulas: server/tools/evaluate_hotspot_metrics.py:263-266
- Train size: 107, Test size: 27, Negatives: 27
- Confusion matrix: TP=19, TN=27, FP=0, FN=8
- Formula 1: Accuracy = $(TP + TN) / (TP + TN + FP + FN)$
- Proof: $(19 + 27) / (19 + 27 + 0 + 8) = 0.8519$ (85.19%)
- Formula 2: Precision = $TP / (TP + FP)$
- Proof: $19 / (19 + 0) = 1.0000$ (100.00%)
- Formula 3: Recall = $TP / (TP + FN)$
- Proof: $19 / (19 + 8) = 0.7037$ (70.37%)
- Formula 4: F1 = $2 * Precision * Recall / (Precision + Recall)$
- Proof: $2 * 1.0000 * 0.7037 / (1.0000 + 0.7037) = 0.8261$ (82.61%)

E) Improved Prediction Accuracy Percentage (with proof)

- Defined baseline: always-majority prediction on balanced test set (27 positives, 27 negatives), accuracy = 50%.
- Reason: evaluation set is balanced, so naive constant predictor yields 27/54 correct.
- Current accuracy = 85.19%
- Improvement (percentage points) = 35.19%
- Relative improvement (%) = $((0.8519 - 0.5000) / 0.5000) * 100 = 70.38\%$

F) Response Time Reduction Percentage (with proof)

- Benchmark method compares two prediction paths on same workload:

- 1) Baseline naive point scan against all training points
- 2) Optimized cluster-centroid scan
- Code proof: server/tools/benchmark_prediction_latency.py
- Baseline avg latency per prediction: 0.221968 ms
- Optimized avg latency per prediction: 0.004234 ms
- Formula: response time reduction % = ((baseline - optimized) / baseline) * 100
- Proof: $((0.221968 - 0.004234) / 0.221968) * 100 = 98.09\%$
- Speedup factor = baseline / optimized = 52.43x

G) Dashboard and Graph Calculations (Simple Explanation + Proof Location)

- 1) Police Dashboard (pages/PoliceDashboard.jsx)
 - - Incidents in 24h and 7d are counted by checking timestamp difference from current time.
 - - Proof: client/src/pages/PoliceDashboard.jsx:145-169
 - - Case status mix buckets: Open/Assigned/Investigating/Closed.
 - - Proof: client/src/pages/PoliceDashboard.jsx:47-58
- 2) Crime Analytics charts (components/CrimeAnalytics.jsx)
 - - State chart = count of records per normalized state.
 - - Proof: client/src/components/CrimeAnalytics.jsx:55-61
 - - Crime type pie = count by type.
 - - Proof: client/src/components/CrimeAnalytics.jsx:63-69
 - - District ranking = top 10 districts by count.
 - - Proof: client/src/components/CrimeAnalytics.jsx:71-81
 - - Time trend line = count by date sorted ascending.
 - - Proof: client/src/components/CrimeAnalytics.jsx:83-94
- 3) Time-Based Analytics (components/TimeBasedAnalytics.jsx)
 - - Hourly graph = 24-bin array, incremented by extracted hour.
 - - Proof: client/src/components/TimeBasedAnalytics.jsx:32-42
 - - Weekly graph = day-of-week counts and peak hour per day.
 - - Proof: client/src/components/TimeBasedAnalytics.jsx:47-67
 - - Year trend percentage = $((\text{secondHalf} - \text{firstHalf}) / \text{firstHalf}) * 100$.
 - - Proof: client/src/components/TimeBasedAnalytics.jsx:104-107
- 4) Officer Performance (components/OfficerPerformance.jsx)
 - - Clearance rate = $\text{solvedCases} / \text{assignedCases} * 100$.
 - - Proof: client/src/components/OfficerPerformance.jsx:81
 - - Overall solved percentage = $\text{solvedCases} / \text{totalCases} * 100$.
 - - Proof: client/src/components/OfficerPerformance.jsx:97
- 5) Crime Trend Predictions (components/CrimeTrendPredictions.jsx)
 - - Daily forecast formula uses 70% day-pattern + 30% recent average adjusted by trend factor.
 - - Proof: client/src/components/CrimeTrendPredictions.jsx:50
 - - Weekly forecast formula scales weekly average by bounded trend and horizon.
 - - Proof: client/src/components/CrimeTrendPredictions.jsx:70
 - - Displayed accuracy in that page is heuristic: $\text{avgConfidence} - 6$ (clamped), not confusion-matrix accuracy.
 - - Proof: client/src/components/CrimeTrendPredictions.jsx:125
- 6) Crime Map (components/CrimeMap.jsx)
 - - Crime cluster marker count groups identical lat,lng points.
 - - Proof: client/src/components/CrimeMap.jsx:457-466
 - - Marker radius is $\min(6 + \text{count} * 2, 25)$, so denser points appear larger.
 - - Proof: client/src/components/CrimeMap.jsx:662
- 7) Real-Time Alerts (components/RealTimeCrimeAlerts.jsx)
 - - Average ETA = mean of numeric etaMinutes values.
 - - Proof: client/src/components/RealTimeCrimeAlerts.jsx:217-220

H) Frequently Asked Viva Questions with Simple Answers

- Q1. Which algorithm is used for hotspot detection?
- Answer: KMeans clustering on latitude-longitude points.
- Q2. Why KMeans?
- Answer: It is fast, stable, and gives clear cluster centers that can be directly shown on the map as hotspots.

- Q3. How is number of clusters chosen?
- Answer: The code tries candidate k values and chooses the one with best silhouette score.
- Q4. What is silhouette score in simple words?
- Answer: It shows how compact each cluster is and how far apart clusters are from each other.
- Q5. What is inertia?
- Answer: It is the sum of squared distances from points to their assigned cluster center. Lower is better, but compare carefully.
- Q6. Why can we not directly trust only dashboard accuracy text?
- Answer: The trend page accuracy is a heuristic display value, not confusion-matrix evaluation.
- Q7. Then where is real Accuracy/Precision/Recall/F1?
- Answer: In evaluation output from temporal holdout script: server/evaluate_metrics_stdout.json.
- Q8. What does Precision=100% mean now?
- Answer: When model predicted hotspot in evaluation, those predictions were correct in this dataset (FP=0).
- Q9. What does Recall=70.37% mean?
- Answer: Model found most, but not all, true hotspots. Some true hotspots were missed (FN=8).
- Q10. How to improve recall?
- Answer: Tune hotspot radius/threshold, add richer features (time/context), and compare DBSCAN/HDBSCAN.
- Q11. What is improved prediction accuracy percentage?
- Answer: Against 50% balanced naive baseline, current 85.19% gives +35.19 points, +70.38% relative lift.
- Q12. How was response-time reduction measured?
- Answer: By benchmarking naive point-scan vs centroid-scan on same query set and repeats.
- Q13. What is current response-time reduction?
- Answer: 98.09% lower average latency per prediction, about 52.43x speedup.
- Q14. Why do we store coverage percent?
- Answer: It shows data quality for training: how many records had usable coordinates/geocodes.
- Q15. Why are some records skipped?
- Answer: Records can be skipped if coordinates are invalid or geocoding fails.
- Q16. What does incidents24h card mean in Police Dashboard?
- Answer: Count of incidents where timestamp difference from now is at most 24 hours.
- Q17. How is incidents7d computed?
- Answer: Same logic, with 7-day window.
- Q18. How is state bar chart computed?
- Answer: Count map by normalized state names.
- Q19. How is district top-10 chart computed?
- Answer: Sort district counts descending and keep first 10.
- Q20. How is crime trend line chart computed?
- Answer: Aggregate count per date then sort by date.
- Q21. How is hourly graph computed?
- Answer: Create 24 bins and increment by parsed hour from crime time/date.
- Q22. How is weekly pattern computed?
- Answer: Count by day-of-week and find each day's peak hour from hour histogram.
- Q23. How is yearly trend percent computed?
- Answer: Compare total crimes in months 7-12 vs 1-6 using percentage change formula.
- Q24. How is officer clearance rate computed?
- Answer: Solved cases divided by assigned cases multiplied by 100.
- Q25. Why may Avg Response Time show N/A in some reports?
- Answer: Current dataset does not provide reliable start-end timestamps for all cases.
- Q26. How is map cluster size shown?
- Answer: Grouped by same coordinates, and marker size grows with count.
- Q27. How is hotspot circle radius decided?
- Answer: Radius scales with hotspot likelihood and user-selected slider bounds.
- Q28. How often does dashboard refresh?
- Answer: Police dashboard refreshes every 20 seconds; several analytics pages refresh every 30 or 60 seconds.
- Q29. Are these numbers reproducible?
- Answer: Yes. Re-run retrain_model.py, evaluate_hotspot_metrics.py, and benchmark_prediction_latency.py.
- Q30. What is your one-line model summary?

- Answer: A KMeans-based geospatial hotspot model with validated quality metrics and measurable latency optimization.

I) Repro Commands (for proof in front of panel)

- From server folder:
- 1) `..\venv\Scripts\python.exe retrain_model.py > retrain_stdout.json`
- 2) `..\venv\Scripts\python.exe tools/evaluate_hotspot_metrics.py > evaluate_metrics_stdout.json`
- 3) `..\venv\Scripts\python.exe tools/benchmark_prediction_latency.py > latency_benchmark_stdout.json`
- 4) `..\venv\Scripts\python.exe tools/generate_complete_review_pdf.py`